



SWEN 221

A surreal, dreamlike landscape. In the center, a person stands on a small, dark, rocky outcrop that appears to be part of a vast, layered sky. The person is looking out over a horizon where a bright, golden light source, possibly a sun or moon, is partially obscured by clouds. The sky is filled with large, billowing clouds in shades of pink, purple, blue, and white. A bright comet streaks across the upper left, and a small, glowing pink nebula is visible in the center. The overall atmosphere is ethereal and otherworldly.

Marco Servetto
CO258

www.youtube.com/MarcoServetto



Reflection



Reflection



The other
side

What is Reflection?

In computer science, reflection is the ability of a program to examine and possibly modify its high level structure at runtime

-- Wikipedia

Reflection is a mechanism by which a program can find out about the capabilities of its objects at runtime, and manipulate the objects whose capabilities it has discovered

-- OOD&P Book

What is reflection?

Reflection in Java

- Java provides `java.lang.Class`

<https://docs.oracle.com/en/java/javase/19/docs/api/java.base/java/lang/Class.html>

- This represents classes
- Each object associated with unique instance of Class
- Can find out about an object by inspecting its Class object

Point

```
public class Point {  
    float x;  
    float y;  
}
```

Left click on “Point” (it became gray)

and then Right click on “Point”

Select “source->generate hashCode() and equals()”

Point

```
public int hashCode() {  
    final int prime = 31;  
    int result = 1;  
    result = prime * result + Float.floatToIntBits(x);  
    result = prime * result + Float.floatToIntBits(y);  
    return result;  
}  
  
public boolean equals(Object obj) {  
    if (this == obj){ return true; }  
    if (obj == null){ return false; }  
    if (this.getClass() != obj.getClass()){ return false; }  
    Point other = (Point) obj;  
    if(Float.floatToIntBits(x) != Float.floatToIntBits(other.x)){ return false; }  
    if(Float.floatToIntBits(y) != Float.floatToIntBits(other.y)){ return false; }  
    return true;  
}
```

Point

```
public int hashCode() {  
    final int prime = 31;  
    int result = 1;  
    result = prime * result + Float.floatToIntBits(x);  
    result = prime * result + Float.floatToIntBits(y);  
    return result;  
}  
  
public boolean equals(Object obj) {  
    if (this == obj){ return true; }  
    if (obj == null){ return false; }  
    if (this.getClass() != obj.getClass()){ return false; }  
    Point other = (Point) obj;  
    if(Float.floatToIntBits(x) != Float.floatToIntBits(other.x)){ return false; }  
    if(Float.floatToIntBits(y) != Float.floatToIntBits(other.y)){ return false; }  
    return true;  
}
```

Point

```
public boolean equals(Object obj) { ...  
    if( this.getClass() != obj.getClass() ){ return false; }  
}
```

Equivalent to

```
public boolean equals(Object obj) { ...  
    Class<? extends Point> myClass = this.getClass();  
    Class<? extends Object> objClass = obj.getClass();  
    if (myClass != objClass){ return false; }  
}
```

Class Objects

```
System.out.println(String.class == "Foo".getClass());
```



**The only
language
feature**

```
System.out.println(String.class.getName());  
// java.lang.String  
System.out.println(String.class.getSimpleName());  
// String
```

Class Objects

```
String s1=  new String("X");  
String s2=  new String("Y");  
Integer i1= new Integer(1);
```

Class Objects

```
String s1=  new String("X");  
String s2=  new String("Y");  
Integer i1= new Integer(1);  
Class<? extends String> c1= s1.getClass();
```

Class Objects

```
String s1=  new String("X");  
String s2=  new String("Y");  
Integer i1= new Integer(1);  
Class<? extends String> c1= s1.getClass();  
Class<String> c2= s2.getClass();
```

Class Objects

```
String s1=  new String("X");  
String s2=  new String("Y");  
Integer i1= new Integer(1);  
Class<? extends String> c1= s1.getClass();  
// Class<String> c2= s2.getClass();//compilation error
```

Class Objects

```
String s1=  new String("X");  
String s2=  new String("Y");  
Integer i1= new Integer(1);  
Class<? extends String> c1= s1.getClass();  
// Class<String> c2= s2.getClass();//compilation error  
Class<String> c2= String.class;
```

Class Objects

```
String s1=  new String("X");
String s2=  new String("Y");
Integer i1= new Integer(1);
Class<? extends String> c1= s1.getClass();
// Class<String> c2= s2.getClass();//compilation error
Class<String> c2= String.class;
Class<? extends Integer> c3= i1.getClass();
```

Class Objects

```
String s1=  new String("X");
String s2=  new String("Y");
Integer i1= new Integer(1);
Class<? extends String> c1= s1.getClass();
// Class<String> c2= s2.getClass();//compilation error
Class<String> c2= String.class;
Class<? extends Integer> c3= i1.getClass();
Class<?> c4= i1.getClass();
```

Class Objects

```
String s1=  new String("X");
String s2=  new String("Y");
Integer i1= new Integer(1);
Class<? extends String> c1= s1.getClass();
// Class<String> c2= s2.getClass();//compilation error
Class<String> c2= String.class;
Class<? extends Integer> c3= i1.getClass();
Class<?> c4= i1.getClass();
assert c1 == c2;
```

Class Objects

```
String s1=  new String("X");
String s2=  new String("Y");
Integer i1= new Integer(1);
Class<? extends String> c1= s1.getClass();
// Class<String> c2= s2.getClass();//compilation error
Class<String> c2= String.class;
Class<? extends Integer> c3= i1.getClass();
Class<?> c4= i1.getClass();
assert c1 == c2;
assert c1 != c3;
```

Class Objects

```
String s1=  new String("X");
String s2=  new String("Y");
Integer i1= new Integer(1);
Class<? extends String> c1= s1.getClass();
// Class<String> c2= s2.getClass();//compilation error
Class<String> c2= String.class;
Class<? extends Integer> c3= i1.getClass();
Class<?> c4= i1.getClass();
assert c1 == c2;
// assert c1 != c3;//compilation error
```

Class Objects

```
String s1=  new String("X");
String s2=  new String("Y");
Integer i1= new Integer(1);
Class<? extends String> c1= s1.getClass();
// Class<String> c2= s2.getClass();//compilation error
Class<String> c2= String.class;
Class<? extends Integer> c3= i1.getClass();
Class<?> c4= i1.getClass();
assert c1 == c2;
// assert c1 != c3;//compilation error
assert c1 != c4;
```

Class Objects

```
String s1=  new String("X");
String s2=  new String("Y");
Integer i1= new Integer(1);
Class<? extends String> c1= s1.getClass();
// Class<String> c2= s2.getClass();//compilation error
Class<String> c2= String.class;
Class<? extends Integer> c3= i1.getClass();
Class<?> c4= i1.getClass();
assert c1 == c2;
// assert c1 != c3;//compilation error
assert c1 != c4;
System.out.println("c1 is a " + c1.getName());
```

Class Objects

```
String s1=  new String("X");
String s2=  new String("Y");
Integer i1= new Integer(1);
Class<? extends String> c1= s1.getClass();
// Class<String> c2= s2.getClass();//compilation error
Class<String> c2= String.class;
Class<? extends Integer> c3= i1.getClass();
Class<?> c4= i1.getClass();
assert c1 == c2;
// assert c1 != c3;//compilation error
assert c1 != c4;
System.out.println("c1 is a " + c1.getName());
// c1 is a java.lang.String
```

Class Objects

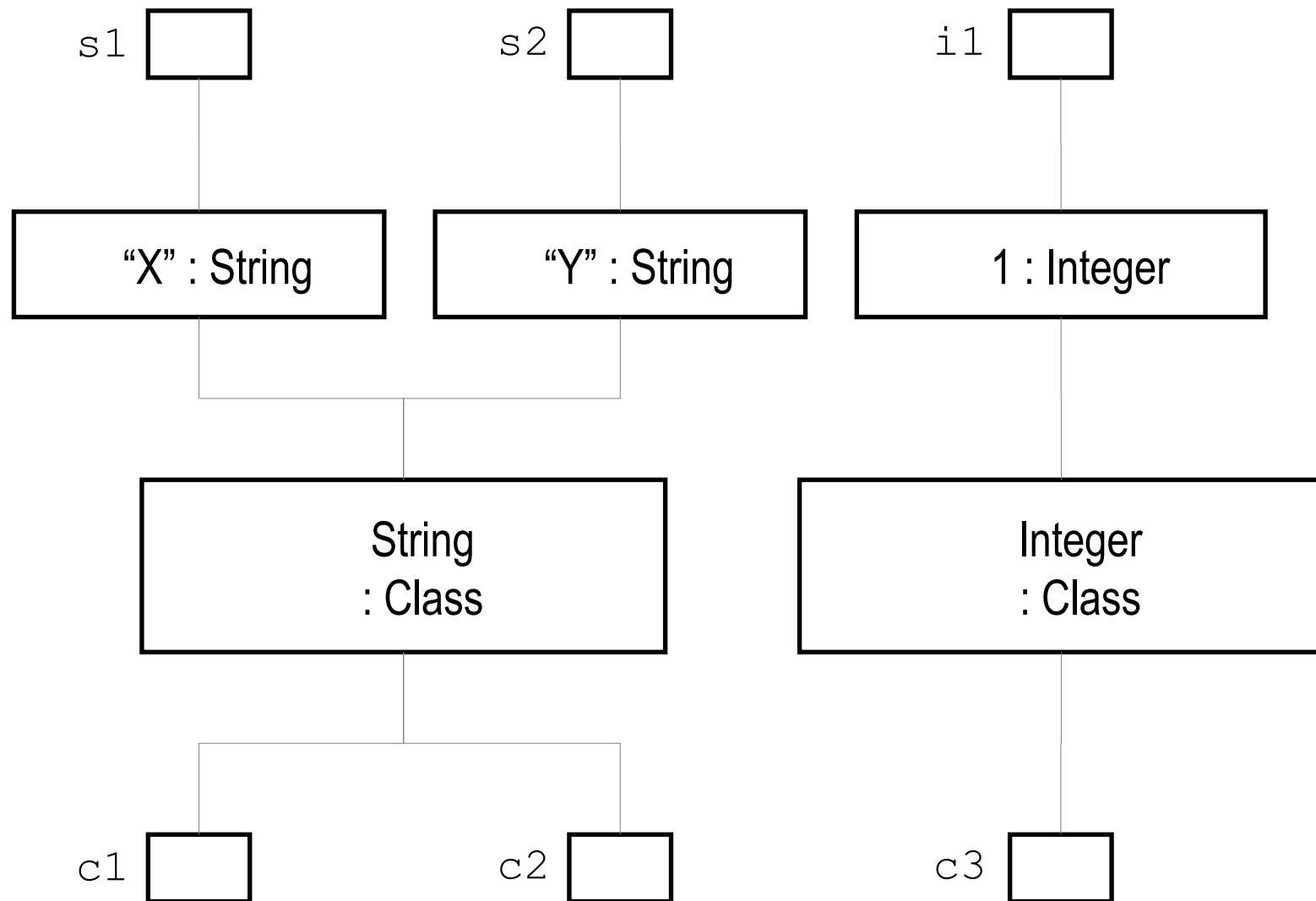
```
String s1=  new String("X");
String s2=  new String("Y");
Integer i1= new Integer(1);
Class<? extends String> c1= s1.getClass();
// Class<String> c2= s2.getClass();//compilation error
Class<String> c2= String.class;
Class<? extends Integer> c3= i1.getClass();
Class<?> c4= i1.getClass();
assert c1 == c2;
// assert c1 != c3;//compilation error
assert c1 != c4;
System.out.println("c1 is a " + c1.getName());
// c1 is a java.lang.String
System.out.println("c3 is a " + c3.getName());
```

Class Objects

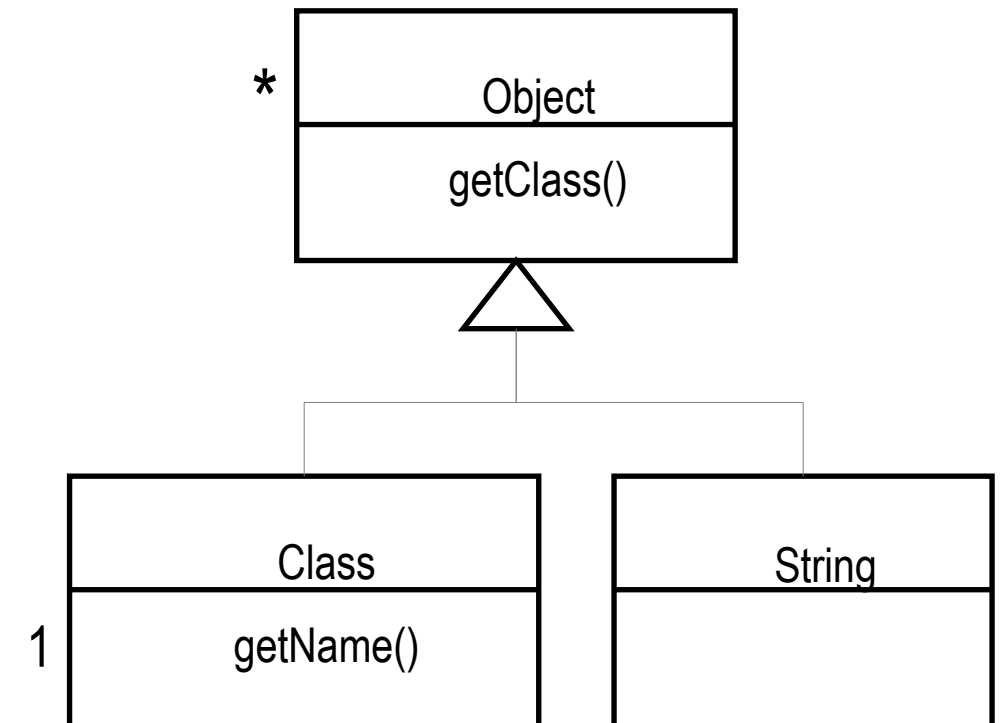
```
String s1=  new String("X");
String s2=  new String("Y");
Integer i1= new Integer(1);
Class<? extends String> c1= s1.getClass();
// Class<String> c2= s2.getClass();//compilation error
Class<String> c2= String.class;
Class<? extends Integer> c3= i1.getClass();
Class<?> c4= i1.getClass();
assert c1 == c2;
// assert c1 != c3;//compilation error
assert c1 != c4;
System.out.println("c1 is a " + c1.getName());
// c1 is a java.lang.String
System.out.println("c3 is a " + c3.getName());
// c3 is a java.lang.Integer
```

Class Objects

Instance Diagram



Class Diagram



Reasoning on levels

- Is this a class ?

```
public class Point{ float x; float y; }
```

- Is this a class ?

```
Class<Point> pointClass= Point.class;
```

- Both 'Point' and 'pointClass' are names referring to 'the class Point', but they exist on different levels of abstraction



Reasoning on levels

- Using the first

```
new ArrayList<Point>()
```

- Using the second

```
foo.bar(pointClass)
```



Q) What gets printed?

```
class SimpleClass {  
    public void aSimpleMethod() { }  
    public void anotherSimpleMethod() { }  
    private void privateMethod() { }  
}
```

...

```
Object o= new SimpleClass();  
Class<?> c= o.getClass();  
Method[] ms= c.getDeclaredMethods();  
for (Method m : ms) {  
    System.out.println("o has method: " + m.getName());  
}
```

Output:



Q) What gets printed?

```
class SimpleClass {  
    public void aSimpleMethod() { }  
    public void anotherSimpleMethod() { }  
    private void privateMethod() { }  
}
```

...

```
Object o= new SimpleClass();  
Class<?> c= o.getClass();  
Method[] ms= c.getDeclaredMethods();  
for (Method m : ms) {  
    System.out.println("o has method: " + m.getName());  
}
```

Output:

```
o has method: aSimpleMethod  
o has method: anotherSimpleMethod  
o has method: privateMethod
```

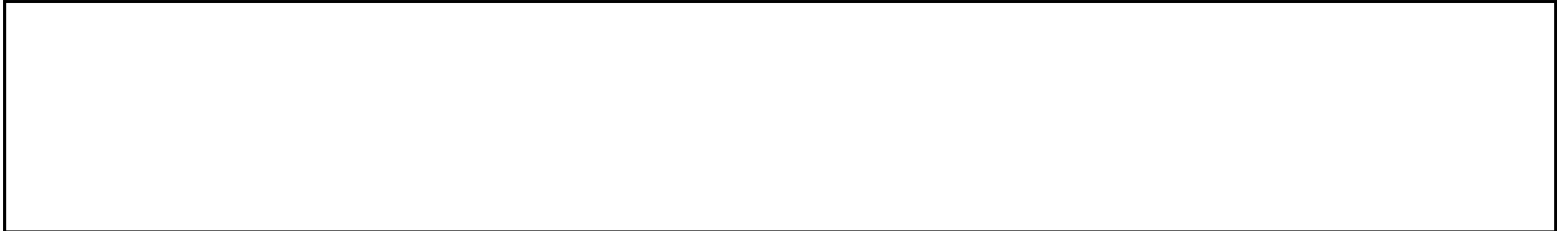
Q) What gets printed?

```
class SimpleClass {  
    public void aSimpleMethod() { }  
    public void anotherSimpleMethod() { }  
    private void privateMethod() { }  
}
```

...

```
Object o= new SimpleClass();  
Class<?> c= o.getClass();  
Method[] ms= c.getMethods();  
for (Method m : ms) {  
    System.out.println("o has method: " + m.getName());  
}
```

Output:



Q) What gets printed?

```
class SimpleClass {  
    public void aSimpleMethod() { }  
    public void anotherSimpleMethod() { }  
    private void privateMethod() { }  
}
```

...

```
Object o= new SimpleClass();  
Class<?> c= o.getClass();  
Method[] ms= c.getMethods();  
for (Method m : ms) {  
    System.out.println("o has method: " + m.getName());  
}
```

Output:

```
o has method: wait  
o has method: equals  
o has method: toString  
o has method: hashCode  
More:    aSimpleMethod, anotherSimpleMethod,  
wait(long,int), wait(long), getClass, notify, notifyAll //NOT 'privateMethod'
```

Metadata

- Reflection gives access to *metadata*
 - That is, data about data
 - In this case, the metadata describes our classes
 - We can find out:
 - What an object's class is
 - What methods that class has (inc. private + protected)
 - What their parameter / return types are
 - What fields that class has (inc. private + protected)
 - What their types are
 - What interfaces the class implements
 - What class it extends from

More than just *metadata*

We can *invoke* methods through reflection as well

```
class SimpleClass {  
    public void aSimpleMethod() {  
        System.out.println("Got called");  
    }  
}  
  
...  
Object o= new SimpleClass();  
Class<?> c= o.getClass();  
try {  
    Method m= c.getMethod("aSimpleMethod");  
    m.invoke(o);  
}  
catch (NoSuchMethodException e) { ... }  
catch (InvocationTargetException e) { ... }  
catch (IllegalAccessException e) { ... }
```

More than just *metadata*

We can *invoke* methods through reflection as well

```
try {
    Method m= c.getMethod("aSimpleMethod");
    m.invoke(o);
}
catch(NoSuchMethodException e)      { throw new Error(e); }
catch(IllegalAccessException e)     { throw new Error(e); }
catch(InvocationTargetException e){
    Throwable ee= e.getCause();
    if(ee instanceof Error err){throw err;}
    if(ee instanceof RuntimeException rte){throw rte;}
    //Otherwise do something...
    //...
}
```

More than just *metadata*

We can *get and set* fields through reflection as well

```
import java.lang.reflect.Field;

public class Test {
    public int afield= 1;
    public static void main(String[] args) {
        Test o= new Test();
        Class<?> c= o.getClass();
        try {
            Field f= c.getField("afield");
            System.out.println("GOT: " + f.get(o));
            f.set(o, 2);
            System.out.println("NOW: " + o.afield);
        }
        catch (NoSuchFieldException e) { throw new Error(e); }
        catch (IllegalAccessException e) { throw new Error(e); }
    }
}
```

Surprising ... ?

What about private fields? Fine Here

```
import java.lang.reflect.Field;

public class Test {
    private int afield= 1;
    public static void main(String[] args) {
        Test o= new Test();
        Class<?> c= o.getClass();
        try {
            Field f= c.getDeclaredField("afield");
            System.out.println("GOT: " + f.get(o));
            f.set(o, 2);
            System.out.println("NOW: " + o.afield);
        }
        catch (NoSuchFieldException e) { throw new Error(e); }
        catch (IllegalAccessException e) { throw new Error(e); }
    }
}
```

Surprising ... ?

What about private fields? Does not work here

```
import java.lang.reflect.Field;
```

```
class Test1 { private int afield= 1;  
    public int getAField() { return this.afield; } }
```

```
public class Test2 {  
    public static void main(String[] args) {  
        Test1 o = new Test1();  
        Class<?> c = o.getClass();  
        try {  
            Field f= c.getDeclaredField("afield");  
  
            System.out.println("GOT: " + f.get(o));  
            f.set(o, 2);  
            System.out.println("NOW: " + o.getAField());  
        }  
        catch (NoSuchFieldException e) { throw new Error(e); }  
        catch (IllegalAccessException e) { throw new Error(e); }  
    }  
}
```

Surprising ... ?

What about private fields? Fine here again!!

```
import java.lang.reflect.Field;

class Test1 { private int afield= 1;
    public int getAField() { return this.afield; } }

public class Test2 {
    public static void main(String[] args) {
        Test1 o = new Test1();
        Class<?> c = o.getClass();
        try {
            Field f= c.getDeclaredField("afield");
            f.setAccessible(true); // !!!!!!!!
            System.out.println("GOT: " + f.get(o));
            f.set(o, 2);
            System.out.println("NOW: " + o.getAField());
        }
        catch (NoSuchFieldException e) { throw new Error(e); }
        catch (IllegalAccessException e) { throw new Error(e); }
    }
}
```

Looks broken! but is just flexible

```
public class RunInSecurity extends SecurityManager{
    public void checkPermission(Permission p) {
        if ("setSecurityManager".equals(p.getName())) {
            throw new SecurityException();
        }
        if ("suppressAccessChecks".equals(p.getName())){
            StackTraceElement[] st = Thread.currentThread().getStackTrace();
            if(.. st ..) { throw new SecurityException(); }
        }
        //Writing a good SecurityManager is harder than this...
    }
    public static void main(String[] args){
        System.setSecurityManager(new RunInSecurity());
        Test2.main(args);
    }
}
```

Looks broken! but is just flexible

```
public class RunInSecurity extends SecurityManager{
    public void checkPermission(Permission p) {
        if ("setSecurityManager".equals(p.getName())) {
            throw new SecurityException();
        }
        if ("suppressAccessChecks".equals(p.getName())){
            StackTraceElement[] st = Thread.currentThread().getStackTrace();
            if(.. st ..) { throw new SecurityException(); }
        }
        //Writing a good SecurityManager is harder than this...
    }
    public static void main(String[] args){
        System.setSecurityManager(new RunInSecurity());
        Test2.main(args);
    }
}
```

This slide made sense in 2022

Whoopsy

- They have deprecated the security manager
- Now the right solution is modules

Reflection



The other
side

Reflection



The other
side is evil

Reflection



The other
side is evil



LETS PLAY A GAME

General idea: twoPlusTwo

```
public interface TwoPlusTwo { int twoPlusTwo(); }  
  
public class ClassInt implements TwoPlusTwo{  
    public/private/final int/Integer twoPlusTwo= 4;  
    public int twoPlusTwo(){ return twoPlusTwo; }  
}
```

What can possibly go wrong?

Let's play a game with modules!

ProtectedModule1.pk1

TwoPlusTwo
(interface)

ClassInt

ClassFInt

ClassInteger

ClassFInteger

ProtectedModule2.pk2

MyClassInt

MyClassFInt

MyClassInteger

MyClassFInteger

```
//module ProtectedLibrary1, public fields
package pk1;
//file TwoPlusTwo.java
public interface TwoPlusTwo { int twoPlusTwo(); }

//file ClassInt.java
public class ClassInt implements TwoPlusTwo{
    public int twoPlusTwo= 4;
    public int twoPlusTwo(){ return twoPlusTwo; }
}

//file ClassFInt.java
public class ClassFInt implements TwoPlusTwo{
    public final int twoPlusTwo= 4;
    public int twoPlusTwo(){ return twoPlusTwo; }
}

//file ClassInteger.java
public class ClassInteger implements TwoPlusTwo{
    public Integer twoPlusTwo= 4;
    public int twoPlusTwo(){ return twoPlusTwo; }
}

//file ClassFInteger.java
public class ClassFInteger implements TwoPlusTwo{
    public final Integer twoPlusTwo= 4;
    public int twoPlusTwo(){ return twoPlusTwo; }
}

//file module-info.java
module ModuleProtectedLibrary1 { exports pk1; }
```

```
//module ProtectedLibrary2, public fields
package pk2;

//file MyClassInt.java
public class MyClassInt implements TwoPlusTwo{
    public int twoPlusTwo= 4;
    public int twoPlusTwo(){ return twoPlusTwo; }
}

//file MyClassFInt.java
public class MyClassFInt implements TwoPlusTwo{
    public final int twoPlusTwo= 4;
    public int twoPlusTwo(){ return twoPlusTwo; }
}

//file MyClassInteger.java
public class MyClassInteger implements TwoPlusTwo{
    public Integer twoPlusTwo= 4;
    public int twoPlusTwo(){ return twoPlusTwo; }
}

//file MyClassFInteger.java
public class MyClassFInteger implements TwoPlusTwo{
    public final Integer twoPlusTwo= 4;
    public int twoPlusTwo(){ return twoPlusTwo; }
}

//file module-info.java
module ModuleProtectedLibrary2 {
    requires ModuleProtectedLibrary1; }
```

```
//module ProtectedLibrary1, public fields
package pk1;
//file TwoPlusTwo.java
public interface TwoPlusTwo { int twoPlusTwo(); }

//file ClassInt.java
public class ClassInt implements TwoPlusTwo{
    public int twoPlusTwo= 4;
    public int twoPlusTwo(){ return twoPlusTwo; }
}

//file ClassFInt.java
public class ClassFInt implements TwoPlusTwo{
    public final int twoPlusTwo= 4;
    public int twoPlusTwo(){ return twoPlusTwo; }
}

//file ClassInteger.java
public class ClassInteger implements TwoPlusTwo{
    public Integer twoPlusTwo= 4;
    public int twoPlusTwo(){ return twoPlusTwo; }
}

//file ClassFInteger.java
public class ClassFInteger implements TwoPlusTwo{
    public final Integer twoPlusTwo= 4;
    public int twoPlusTwo(){ return twoPlusTwo; }
}

//file module-info.java
module ModuleProtectedLibrary1 { exports pk1; }
```

```
//module ProtectedLibrary2, private fields
package pk2;

//file MyClassInt.java
public class MyClassInt implements TwoPlusTwo{
    private int twoPlusTwo= 4;
    public int twoPlusTwo(){ return twoPlusTwo; }
}

//file MyClassFInt.java
public class MyClassFInt implements TwoPlusTwo{
    private final int twoPlusTwo= 4;
    public int twoPlusTwo(){ return twoPlusTwo; }
}

//file MyClassInteger.java
public class MyClassInteger implements TwoPlusTwo{
    private Integer twoPlusTwo= 4;
    public int twoPlusTwo(){ return twoPlusTwo; }
}

//file MyClassFInteger.java
public class MyClassFInteger implements TwoPlusTwo{
    private final Integer twoPlusTwo= 4;
    public int twoPlusTwo(){ return twoPlusTwo; }
}

//file module-info.java
module ModuleProtectedLibrary2 {
    requires ModuleProtectedLibrary1; }
```

```
//module ProtectedLibrary1, private fields
package pk1;
//file TwoPlusTwo.java
public interface TwoPlusTwo { int twoPlusTwo(); }

//file ClassInt.java
public class ClassInt implements TwoPlusTwo{
    private int twoPlusTwo= 4;
    public int twoPlusTwo(){ return twoPlusTwo; }
}

//file ClassFInt.java
public class ClassFInt implements TwoPlusTwo{
    private final int twoPlusTwo= 4;
    public int twoPlusTwo(){ return twoPlusTwo; }
}

//file ClassInteger.java
public class ClassInteger implements TwoPlusTwo{
    private Integer twoPlusTwo= 4;
    public int twoPlusTwo(){ return twoPlusTwo; }
}

//file ClassFInteger.java
public class ClassFInteger implements TwoPlusTwo{
    private final Integer twoPlusTwo= 4;
    public int twoPlusTwo(){ return twoPlusTwo; }
}

//file module-info.java
module ModuleProtectedLibrary1 { exports pk1; }
```

```
//module ProtectedLibrary2, private fields
package pk2;

//file MyClassInt.java
public class MyClassInt implements TwoPlusTwo{
    private int twoPlusTwo= 4;
    public int twoPlusTwo(){ return twoPlusTwo; }
}

//file MyClassFInt.java
public class MyClassFInt implements TwoPlusTwo{
    private final int twoPlusTwo= 4;
    public int twoPlusTwo(){ return twoPlusTwo; }
}

//file MyClassInteger.java
public class MyClassInteger implements TwoPlusTwo{
    private Integer twoPlusTwo= 4;
    public int twoPlusTwo(){ return twoPlusTwo; }
}

//file MyClassFInteger.java
public class MyClassFInteger implements TwoPlusTwo{
    private final Integer twoPlusTwo= 4;
    public int twoPlusTwo(){ return twoPlusTwo; }
}

//file module-info.java
module ModuleProtectedLibrary2 {
    requires ModuleProtectedLibrary1; }
```

The play method

```
public class LetsPlayAGame {  
  
    static void play(TwoPlusTwo o) {  
        try { playAux(o); }  
        catch(Throwable e){ System.out.println("Failure "+e);}  
    }  
  
    static void playAux(TwoPlusTwo o)  
        throws NoSuchFieldException, SecurityException,  
            IllegalArgumentException, IllegalAccessException {  
        Field f= o.getClass().getDeclaredField("twoPlusTwo");  
        f.setAccessible(true);  
        f.set(o,5);  
        System.out.println(o.getClass().getSimpleName() + ": " + o.twoPlusTwo());  
    }  
    ...  
}
```

```
public class LetsPlayAGame {//Case with all public fields
```

The Main!

```
    static void play(TwoPlusTwo o) {...}
```

```
public static void main(String[] args) {  
    play(new MyClassInt());           //My module, int field           MyClassInt: ???  
    play(new MyClassFInt());          //My module, final int field      ???  
    play(new MyClassInteger());       //My module, Integer field       ???  
    play(new MyClassFInteger());      //My module,final Integer field  ???  
  
    play(new ClassInt());             //other module, int field        ???  
    play(new ClassFInt());            //other module, final int field  ???  
    play(new ClassInteger());         //other module, Integer field    ???  
    play(new ClassFInteger());        //other module, final int field  ???  
}  
}
```

```
public class LetsPlayAGame {//Case with all public fields
```

```
    static void play(TwoPlusTwo o) {...}
```

```
    public static void main(String[] args) {
```

```
        play(new MyClassInt());           //My module, int field           MyClassInt: 5
```

```
        play(new MyClassFInt());          //My module, final int field      ???
```

```
        play(new MyClassInteger());       //My module, Integer field        ???
```

```
        play(new MyClassFInteger());      //My module, final Integer field  ???
```

```
        play(new ClassInt());             //other module, int field        ???
```

```
        play(new ClassFInt());            //other module, final int field    ???
```

```
        play(new ClassInteger());         //other module, Integer field      ???
```

```
        play(new ClassFInteger());        //other module, final int field    ???
```

```
    }
```

```
}
```

The Main!

```
public class LetsPlayAGame {//Case with all public fields
```

```
    static void play(TwoPlusTwo o) {...}
```

```
    public static void main(String[] args) {
```

```
        play(new MyClassInt());           //My module, int field           MyClassInt: 5  
        play(new MyClassFInt());          //My module, final int field      MyClassFInt: 4  
        play(new MyClassInteger());       //My module, Integer field        ???  
        play(new MyClassFInteger());      //My module,final Integer field    ???
```

```
        play(new ClassInt());             //other module, int field        ???  
        play(new ClassFInt());            //other module, final int field    ???  
        play(new ClassInteger());         //other module, Integer field     ???  
        play(new ClassFInteger());        //other module, final int field    ???
```

```
    }
```

```
}
```

The Main!

```
public class LetsPlayAGame {//Case with all public fields
```

```
    static void play(TwoPlusTwo o) {...}
```

```
    public static void main(String[] args) {
```

```
        play(new MyClassInt()); //My module, int field
```

```
        play(new MyClassFInt()); //My module, final int field
```

```
        play(new MyClassInteger()); //My module, Integer field
```

```
        play(new MyClassFInteger()); //My module, final Integer field
```

```
        play(new ClassInt()); //other module, int field
```

```
        play(new ClassFInt()); //other module, final int field
```

```
        play(new ClassInteger()); //other module, Integer field
```

```
        play(new ClassFInteger()); //other module, final int field
```

```
    }
```

```
}
```

The Main!

MyClassInt: 5

MyClassFInt: 4

MyClassInteger: 5

???

???

???

???

???

```
public class LetsPlayAGame {//Case with all public fields
```

```
    static void play(TwoPlusTwo o) {...}
```

```
    public static void main(String[] args) {
```

```
        play(new MyClassInt()); //My module, int field
```

MyClassInt: 5

```
        play(new MyClassFInt()); //My module, final int field
```

MyClassFInt: 4

```
        play(new MyClassInteger()); //My module, Integer field
```

MyClassInteger: 5

```
        play(new MyClassFInteger()); //My module, final Integer field
```

MyClassFInteger: 5

```
        play(new ClassInt()); //other module, int field
```

???

```
        play(new ClassFInt()); //other module, final int field
```

???

```
        play(new ClassInteger()); //other module, Integer field
```

???

```
        play(new ClassFInteger()); //other module, final int field
```

???

```
    }
```

```
}
```

The Main!

```
public class LetsPlayAGame {//Case with all public fields
```

```
    static void play(TwoPlusTwo o) {...}
```

```
    public static void main(String[] args) {
```

```
        play(new MyClassInt()); //My module, int field
```

MyClassInt: 5

```
        play(new MyClassFInt()); //My module, final int field
```

MyClassFInt: 4

```
        play(new MyClassInteger()); //My module, Integer field
```

MyClassInteger: 5

```
        play(new MyClassFInteger()); //My module, final Integer field
```

MyClassFInteger: 5

```
        play(new ClassInt()); //other module, int field
```

ClassInt: 5

```
        play(new ClassFInt()); //other module, final int field
```

???

```
        play(new ClassInteger()); //other module, Integer field
```

???

```
        play(new ClassFInteger()); //other module, final int field
```

???

```
    }
```

```
}
```

The Main!

```
public class LetsPlayAGame {//Case with all public fields
```

```
    static void play(TwoPlusTwo o) {...}
```

```
    public static void main(String[] args) {
```

```
        play(new MyClassInt()); //My module, int field
```

```
        play(new MyClassFInt()); //My module, final int field
```

```
        play(new MyClassInteger()); //My module, Integer field
```

```
        play(new MyClassFInteger()); //My module, final Integer field
```

```
        play(new ClassInt()); //other module, int field
```

```
        play(new ClassFInt()); //other module, final int field
```

```
        play(new ClassInteger()); //other module, Integer field
```

```
        play(new ClassFInteger()); //other module, final int field
```

```
    }
```

```
}
```

The Main!

MyClassInt: 5

MyClassFInt: 4

MyClassInteger: 5

MyClassFInteger:5

ClassInt: 5

ClassFInt: 4

???

???

```
public class LetsPlayAGame {//Case with all public fields
```

```
    static void play(TwoPlusTwo o) {...}
```

```
    public static void main(String[] args) {
```

```
        play(new MyClassInt()); //My module, int field
```

```
        play(new MyClassFInt()); //My module, final int field
```

```
        play(new MyClassInteger()); //My module, Integer field
```

```
        play(new MyClassFInteger()); //My module, final Integer field
```

```
        play(new ClassInt()); //other module, int field
```

```
        play(new ClassFInt()); //other module, final int field
```

```
        play(new ClassInteger()); //other module, Integer field
```

```
        play(new ClassFInteger()); //other module, final int field
```

```
    }
```

```
}
```

The Main!

MyClassInt: 5

MyClassFInt: 4

MyClassInteger: 5

MyClassFInteger: 5

ClassInt: 5

ClassFInt: 4

ClassInteger: 5

???

```
public class LetsPlayAGame {//Case with all public fields
```

```
    static void play(TwoPlusTwo o) {...}
```

```
    public static void main(String[] args) {
```

```
        play(new MyClassInt()); //My module, int field
```

```
        play(new MyClassFInt()); //My module, final int field
```

```
        play(new MyClassInteger()); //My module, Integer field
```

```
        play(new MyClassFInteger()); //My module, final Integer field
```

```
        play(new ClassInt()); //other module, int field
```

```
        play(new ClassFInt()); //other module, final int field
```

```
        play(new ClassInteger()); //other module, Integer field
```

```
        play(new ClassFInteger()); //other module, final int field
```

```
    }
```

```
}
```

The Main!

MyClassInt: 5

MyClassFInt: 4

MyClassInteger: 5

MyClassFInteger: 5

ClassInt: 5

ClassFInt: 4

ClassInteger: 5

ClassFInteger: 5

```
public class LetsPlayAGame {//other module public, but this private
```

```
    static void play(TwoPlusTwo o) {...}
```

The Main!

```
public static void main(String[] args) {
```

```
    play(new MyClassInt());           //My module, int field           MyClassInt: 5  
    play(new MyClassFInt());          //My module, final int field      MyClassFInt: 4  
    play(new MyClassInteger());       //My module, Integer field      MyClassInteger: 5  
    play(new MyClassFInteger());      //My module,final Integer field  MyClassFInteger:5
```

```
    play(new ClassInt());             //other module, int field       ClassInt: 5  
    play(new ClassFInt());            //other module, final int field  ClassFInt: 4  
    play(new ClassInteger());         //other module, Integer field    ClassInteger: 5  
    play(new ClassFInteger());        //other module, final int field  ClassFInteger: 5
```

```
}
```

```
}
```

No change!

The Main!

```
public class LetsPlayAGame {//Case with all fields private
```

```
    static void play(TwoPlusTwo o) {...}
```

```
public static void main(String[] args) {
```

```
    play(new MyClassInt());           //My module, int field      MyClassInt: 5  
    play(new MyClassFInt());          //My module, final int field MyClassFInt: 4  
    play(new MyClassInteger());       //My module, Integer field   MyClassInteger: 5  
    play(new MyClassFInteger());      //My module, final Integer field MyClassFInteger:5
```

```
    play(new ClassInt()); //other module, ..      ???
```

```
    play(new ClassFInt()); //other module, ..      ???
```

```
    play(new ClassInteger()); //other module, ..   ???
```

```
    play(new ClassFInteger()); //other module, ..  ???
```

```
}
```

```
}
```

The Main!

```
public class LetsPlayAGame {//Case with all fields private
```

```
    static void play(TwoPlusTwo o) {...}
```

```
public static void main(String[] args) {
```

```
    play(new MyClassInt());           //My module, int field      MyClassInt: 5  
    play(new MyClassFInt());          //My module, final int field  MyClassFInt: 4  
    play(new MyClassInteger());       //My module, Integer field    MyClassInteger: 5  
    play(new MyClassFInteger());      //My module, final Integer field MyClassFInteger:5
```

```
    play(new ClassInt());//other module, ..      InaccessibleObjectException  
    play(new ClassFInt());//other module, ..      InaccessibleObjectException  
    play(new ClassInteger());//other module, ..   InaccessibleObjectException  
    play(new ClassFInteger());//other module, ..  InaccessibleObjectException
```

```
}
```

```
}
```

The Errors!

Failure java.lang.reflect.InaccessibleObjectException:
Unable to make field private int pk1.ClassInt.twoPlusTwo
accessible:
module ModuleProtectedLibrary1 does not "opens pk1" to module ModuleProtectedLibrary2

Failure java.lang.reflect.InaccessibleObjectException:
Unable to make field private final int pk1.ClassFInt.twoPlusTwo
accessible:
module ModuleProtectedLibrary1 does not "opens pk1" to module ModuleProtectedLibrary2

Failure java.lang.reflect.InaccessibleObjectException:
Unable to make field private final java.lang.Integer pk1.ClassInteger.twoPlusTwo
accessible:
module ModuleProtectedLibrary1 does not "opens pk1" to module ModuleProtectedLibrary2

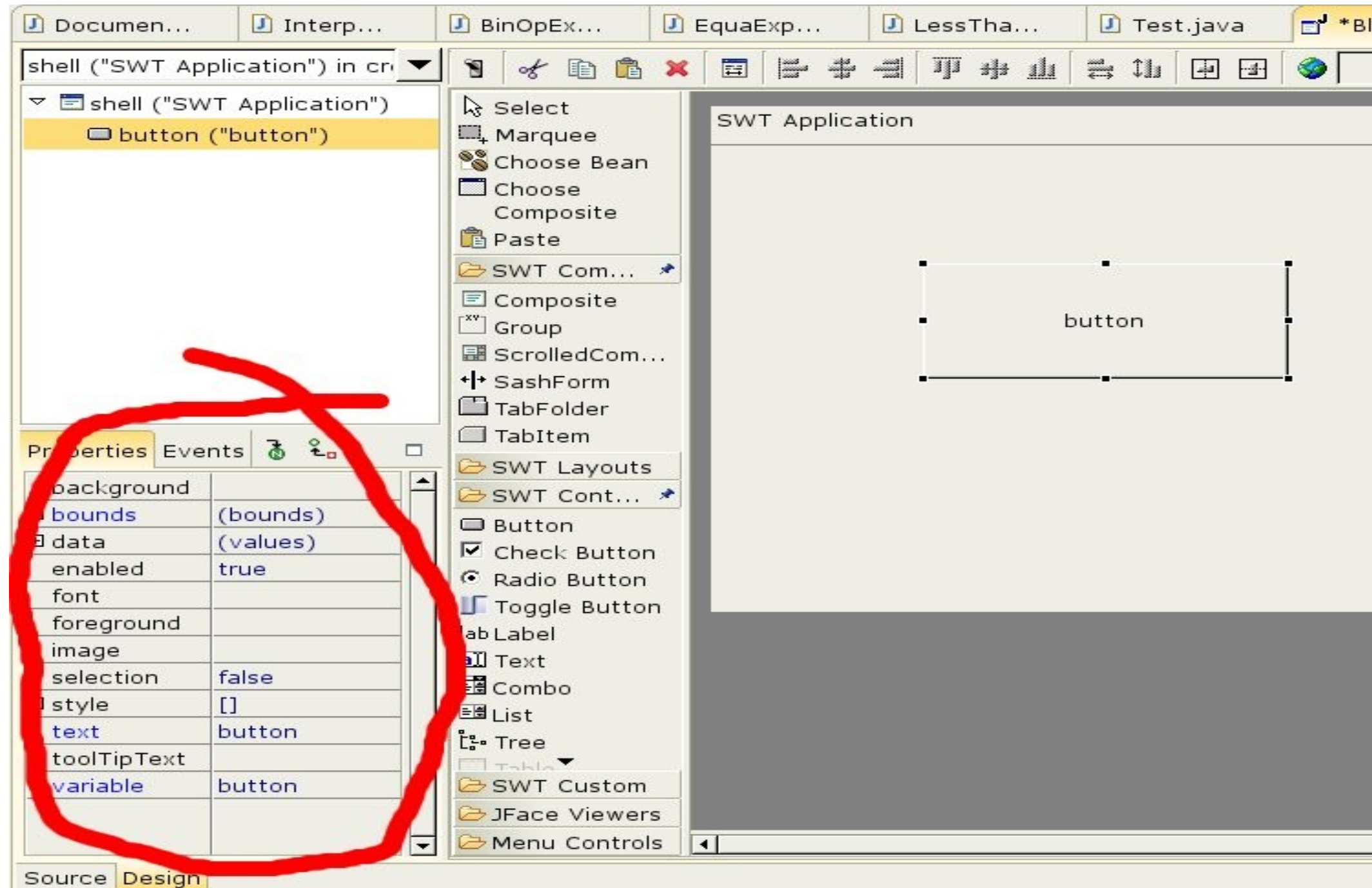
Failure java.lang.reflect.InaccessibleObjectException:
Unable to make field private final java.lang.Integer pk1.ClassFInteger.twoPlusTwo
accessible:
module ModuleProtectedLibrary1 does not "opens pk1" to module ModuleProtectedLibrary2

Note: java.lang.reflect.InaccessibleObjectException extends RuntimeException

Why is reflection useful?

- Anonymous objects
 - Objects whose class is not known at compile time
 - Reflection enables us to use anonymous objects
- Example – GUI builder
 - A classic use of reflection
 - A GUI builder displays objects
 - User can see what attributes it supports
 - E.g. colour, size, position, text, icons etc
 - We want to add new GUI components whenever we like

Example – SWT designer



Why is reflection useful?

- `Class<?> c= Class.forName("myPName.MyClass");`
 - Obtain a class object from a string!
 - Unlock your imagination
 - Example, in the first version of the Web Assessment Tool:

```
Class<?> c= Class.forName("questions.QuestionW0_0");  
Question q= (Question)c  
    .getDeclaredConstructor()  
    .newInstance()
```

- now I have a Question object, and I can use it without any more reflection.
- Similar patterns are used to do plug-in loading and automatically upgradable programs.